Tuesday, September 9, 2025

LEC 02 - Introduction to Assembly

RISC-V Instruction Set

- Used throughout the course
- Applications in consumer electronics, network/storage equipment, cameras, printers, etc.

Example

C Code

```
f = (i + 5) + (g * 4);
```

RISC-V Code

```
addi x5, x18, 5          # temp t0 = i + 5
slli x6, x19, 2          # temp t1 = g * 4
add x7, x5, x6           # f = t0 + t1
```

- I-format instructions (Immediate format) (ex. `addi`, `slli`) use an immediate (constant) value
- R-format instructions (Register format) (ex. `add`, `mv`) use the value of another register

---

In-Class Activity

1. Calculate the expression $(1024 - 512) - (256 - 128)$ and store the result in $x5$
2. Calculate the expression $(888/8 - 123 * 4) * 2$ and store the result in $x5$

Representing Instructions

- Instructions are encoded as binary numbers
  - Each instruction has a specific format
  - This representation is called machine code
- RISC-V instructions are:
  - Encoded as 32-bit instruction words
  - Categorized into a small number of formats based on their operands (registers vs. immediate)
  - Structured to be regular

Hexadecimal Representation

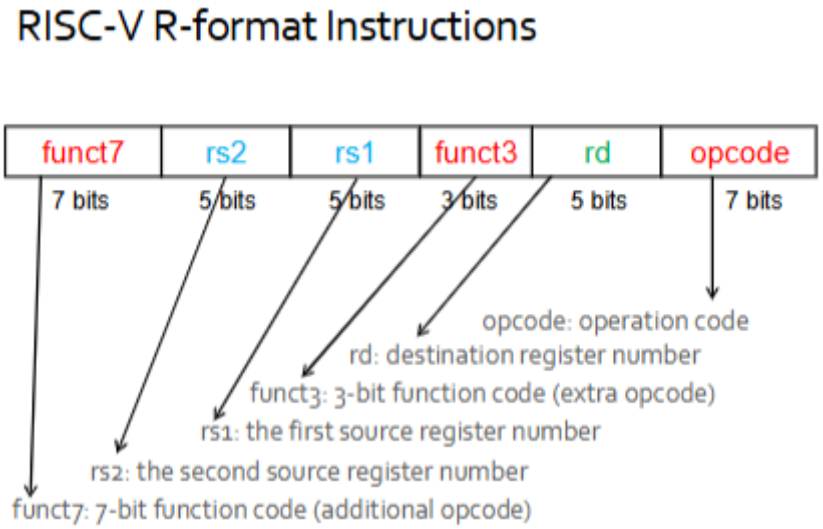
- Base 16

Tuesday, September 9, 2025

LEC 02 - Introduction to Assembly

- Compact representation of bit strings
- 4 bits per hex digit

0	0000	**4**	0100	**8**	1000	**c**	1100
1	0001	**5**	0101	**9**	1001	**d**	1101
2	0010	**6**	0110	**a**	1010	**e**	1110
3	0011	**7**	0111	**b**	1011	**f**	1111



Example

Convert the following RISC-V instruction into the binary format
add x9, x20, x21

0000000	10101	10100	000	01001	0110011

RISC-V I-Format Instructions

- Immediate arithmetic and load instructions
  - rs1: source or base address register number
  - immediate: constant operand of offset added to base address
    - 25 complement, sign extended (week 4 content)
- Conflict: different instructions need different fields, placing all the fields in one format wastes space

Tuesday, September 9, 2025

LEC 02 - Introduction to Assembly

- Compromise: RISC-V designers decided to keep all instructions the same length, requiring distinct instruction formats for different kinds of instructions

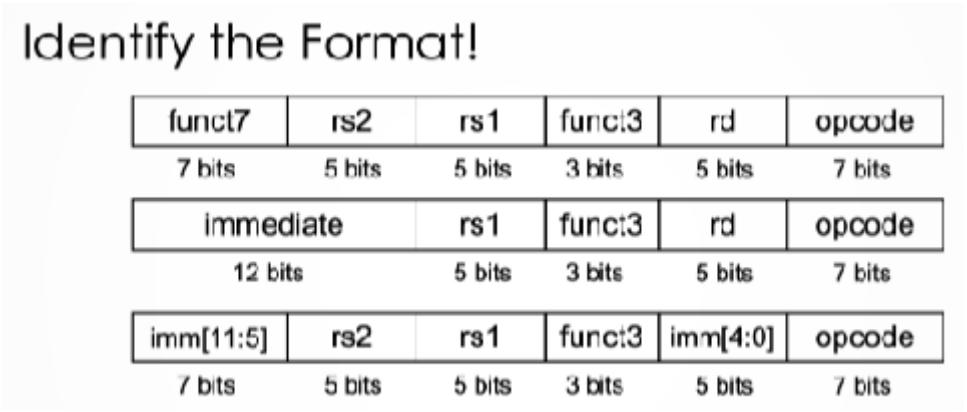
immediate	rs1	funct3	rd	opcode
12 bits	5 bits	3 bits	5 bits	7 bits

RISC-V S-Format Instructions

- Different immediate format for store instructions
  - rs1: base address register number
  - Rs2: source operand register number
  - immediate: offset added to base address
    - Split so that the rs1 and rs2 fields are always in the same place

---

Example



1) R-Format
2) I-Format
3) S-Format

Machine Code Translation

- We can now take an example all the way from what the programmer writes to what the computer executes

```
A[30] = h + A[30] + 1
```

Tuesday, September 9, 2025

LEC 02 - Introduction to Assembly

```
lw x7, 120 # Temporary reg x7 gets assigned A[30] (30 * 4 = 120)
add x7, x21, x7 # Temporary reg x7 gets h + A[30]
addi x7, x7, 1 # Temporary reg x7 gets h + A[30] + 1
sw x7, 120 # Stores h + A[30] + 1 back into A[30]
```

000001111000 10010 010 00111 0000011

RISC-V ISA

- A reduced instruction set computer
  - Instructions are simple and minimal
  - Complex instructions are built from simple ones
- A *load-store* architecture
  - Data can move from register to register or from register to memory
  - Operations cannot be performed *in* memory

The Stored-Program Concept

- RISC-V implements the stored program concept
  - Another big idea in architecture
  - Memory contains binary data
  - Programs are represented as binary data in a specific format
    - Any computer that uses a specific instruction set can accept a binary, rather than having to rebuild the program from source

---

Metrics to Evaluate an ISA

- If the ISA has few registers, they will have to be spilled (saved to memory) often
- If you have many registers, you need more bits to address them
- Various design, hardware, and performance constraints lead us to develop metrics for evaluating ISA

ISA Design Principles

- Simplicity favours regularity
  - All arithmetic operations have this form
    - Two sources and one destination
    - Ex. `add a, b, c` ⇒ `a = b + c`
  - All instructions have opcodes in the same place

LEC 02 - Introduction to Assembly

        - Regularity makes implementation simpler, so more efficient decoders and compilers can be written
- Smaller is faster (ease of compilation)
    - A larger number of registers may increase the clock cycle time because it increases the processor size
    - Larger instructions (due to register addressing) require more width (wires) to transmit data
    - More complex instructions are difficult to optimize effectively
- RISC-V has a 32-element register file
    - Used only for frequently accessed data
    - Send other data to memory according to the memory hierarchy
- Good design demands good compromises to ensure completeness
    - Different formats make decoding complicated, but allows uniform 32-bit instructions
    - Keep formats as similar as possible
- Meeting these principles supports the design of instruction sets for pipelining (week 6)

Registers vs. Memory

- Registers are faster to access than memory
- Operating on memory data requires loads and stores
    - More instructions to be executed
- Compilers use registers for variables as much as possible
    - Only spill to memory for less frequently used variables
    - Register optimization is important
- Some hardware optimizes register usage

Immediate Operands

- Constant data specified in an instruction…
    - Saves a memory load
    - Makes the common case fast
        - Small constants are common
        - Immediate operand avoids a load instruction
        - Justifies a design compromise